

CLOCKS, ORDERING & SEQUENCE01

Sequence number — monotonic counter giving a total order to events/writes.
▲ A single global sequencer is a throughput ceiling & SPOF; shard it or you bottleneck.

Monotonic clock — never goes backwards; for measuring elapsed time.
▲ Never use wall-clock for durations — NTP steps can make it jump back; use monotonic.

LSN (Log Sequence Number) — position in the WAL; identifies each change.
▲ Replicas track LSN to measure lag; a lagging LSN = stale reads on that replica.

Logical clock / Lamport — orders events without synced wall clocks.
▲ Gives "happened-before," not real time; can't compare unrelated events.

Vector clock — per-node counters detecting concurrent writes.
▲ Grows with node count; needs pruning, and the app still has to merge conflicts.

Hybrid logical clock (HLC) — logical clock bounded to physical time.
▲ Best of both, but still trusts NTP within a bound; not a substitute for TrueTime.

Clock skew — node clocks disagree.
▲ Breaks timestamp ordering & LWW; Spanner needs TrueTime/atomic clocks to dodge it.

CACHE & WRITE STRATEGIES02

Cache-aside (lazy) — app reads cache, on miss loads DB & populates.
▲ First request is always a miss; stale data until TTL — pair with invalidation.

Read-through — cache itself loads from DB on miss.
▲ Cache becomes a dependency; if it's down, reads stop.

Write-through — write to cache + DB synchronously.
▲ Every write pays both latencies; consistent but slower writes.

Write-behind (write-back) — write cache now, flush to DB async.
▲ Fast writes, but unflushed data is lost if the cache dies — durability risk.

Refresh-ahead — proactively reload hot keys before they expire.
▲ Wastes work refreshing keys nobody reads; only worth it for known-hot keys.

Last-write-wins (LWW) — newest timestamp wins on conflict.
▲ Silently discards the loser's data; under clock skew the "wrong" write can win.

TTL & eviction (LRU/LFU) — expire by time; evict by recency/frequency under pressure.
▲ Uniform TTLs expire together → stampede; jitter them. LRU thrashes on scan-heavy loads.

Cache invalidation — remove/update stale entries on write.
▲ The hard problem: write-then-invalidate has a race window; consider versioned keys.

Negative caching — cache "not found" results too.
▲ Stops penetration attacks hammering the DB for missing keys; use a short TTL.

LOCKING & CONSISTENCY03

Optimistic lock (version) — read version, write only if unchanged (CAS/ETag).
▲ Cheap under low contention; under high contention it's all retries & wasted work.

Pessimistic lock — lock the row up front for the duration.
▲ Held across slow user steps = poor concurrency + deadlock risk.

ACID — atomicity, consistency, isolation, durability.
▲ Doesn't survive across services/shards — that's where you switch to sagas.

Read-your-writes — a user always sees their own last write.
▲ Easy to break with read replicas; route the user to the leader briefly after a write.

Idempotency key — client token making a retried op safe.
▲ Must handle the concurrent-retry race (unique constraint) or you double-apply.

Isolation levels — read-committed → repeatable-read → serializable.
▲ Higher = fewer anomalies but more locking/aborts; know your DB's default (often RC).

Quorum (R + W > N) — overlapping read/write sets guarantee freshness.
▲ Tunable, but R=W=1 is fast & stale; the math only bounds staleness, not conflicts.

TRAFFIC SHAPING & ROUTING04

Cell / POD architecture — independent stack per user subset.
▲ Shrinks blast radius, but cross-cell ops & rebalancing are hard; routing layer is critical.

Multi-region lag — async replication delay between regions.
▲ A region failover can lose the un-replicated tail; reads in region B are stale.

Anycast / DNS PoP — same IP, routed to nearest point of presence.
▲ DNS TTL caching means failover isn't instant — clients hold stale records.

GeoDNS / latency routing — DNS answers by client location/RTT.
▲ Resolver location ≠ user location; can misroute users behind big resolvers.

L4 vs L7 load balancing — route on TCP/IP vs HTTP content.
▲ L7 enables path/header routing but terminates TLS & costs more CPU than L4 passthrough.

LB algorithms — round-robin, least-connections, P2C (power-of-two).
▲ Round-robin ignores load → hits slow nodes; P2C avoids the herd of "least-loaded."

Consistent-hash routing — sticky map of key→ node for cache affinity.
▲ One node leaving reshuffles a slice; without vnodes the load lands unevenly.

Backpressure — downstream signals "slow down" upstream.
▲ Without it, queues grow unbounded → OOM; bound queues & reject early instead.

Load shedding — drop excess requests to protect the rest.
▲ Shed cheap/low-priority first; shedding everything equally just spreads the pain.

Circuit breaker — stop calling a failing dependency (open/half-open).
▲ Too-sensitive trips cause needless outages; tune per dependency, add a fallback.

Admission control — gate requests at the door to protect SLOs.
▲ Must shed low-priority first & signal clients (429) or rejected load just retries.

Timeout + deadline propagation — cap wait; pass remaining budget down the chain.
▲ No timeout = threads pile up on a hung call; propagate or downstream wastes work on a dead request.

Hedged request — fire a 2nd copy if the 1st is slow; take first to return.
▲ Cuts tail latency but adds load; cap with a tight P95 trigger or you double traffic.

Connection draining — stop new conns, let in-flight finish before shutdown.
▲ Skip it and a deploy kills live requests; set a drain timeout under the LB's.

Priority / QoS lanes — separate queues so critical traffic isn't starved.
▲ One fat batch job can starve interactive traffic; isolate lanes & weight them.

DNS poisoning — attacker injects forged DNS records.
▲ A security threat, not a routing tool; mitigate with DNSSEC.

TOP-LINE METRICS05

P50 / P99 / P999 — latency at the 50th/99th/99.9th percentile.
▲ P99 is what your heaviest users feel; optimizing the mean hides the tail.

Mean vs tail — average vs the slow extremes.
▲ Fan-out makes tail dominate: 100 calls at P99=1% → ~63% of requests hit a slow one.

Uptime / availability — % of time the service meets its SLO.
▲ Define it by successful requests, not "server is pingable" — those differ a lot.

Downtime budget — allowed unavailability (99.9% ≈ 43 min/mo).
▲ Each extra nine is ~10× cost; don't promise nines the business won't pay for.

Error budget — the SLO's allowed failure, spent on risk/velocity.
▲ Burn it too fast → freeze releases; it's a contract between product & ops.

Saturation — how "full" the busiest resource is (USE method).
▲ Latency stays flat then hits a knee near saturation — watch the trend, not the cliff.

CAPACITY & ESTIMATION05B

QPS / peak-to-average — request rate; peaks run 2–10× the average.
▲ Size for peak, not average, or you fall over at the worst moment.

Little's Law — concurrency = arrival rate × latency.
▲ Cut latency and you cut needed concurrency/threads; the lever is often latency, not boxes.

Headroom — spare capacity kept above expected peak.
▲ Running hot leaves no room for failover load — a dead node dumps its traffic on you.

Retries (the dark side) — re-sending on failure to ride out transients.
 ▲ Retries multiply load on an already-struggling service; bound them + use a retry budget.

Thundering herd — mass simultaneous requests on one resource.
 ▲ Cache-miss stampede on a hot key crushes the DB; coalesce requests + jitter TTLs.

Metastable failure — system stays down after the trigger clears.
 ▲ Retry storms keep it pinned; you must shed load to escape, not just wait.

Hot partition / key skew — one shard takes disproportionate load.
 ▲ A celebrity/viral key melts one node; salt the key or split it.

Cascading failure — one component's failure topples dependents.
 ▲ Synchronous call chains propagate it; bulkheads + timeouts contain it.

Gray failure — degraded-but-alive; passes health checks while failing real work.
 ▲ The worst kind — monitoring says green; measure from the caller's view, not self-reported.

Coordinated omission — load tools under-count latency by pausing when the system stalls.
 ▲ Your P99 looks great because the worst samples were never taken; correct for it.

DEPLOYMENT & RELEASE

Blue-green — two identical envs; flip traffic to the new one.
 ▲ Instant rollback, but double the infra; DB schema must serve both at once.

Canary — release to a small % first, watch metrics, then ramp.
 ▲ Needs good metrics & auto-rollback; a slow canary delays the rollout.

Rolling — replace instances batch by batch.
 ▲ Old + new run together mid-roll; both versions must be compatible.

Feature flag — decouple deploy from release; toggle at runtime.
 ▲ Flags accumulate into debt; stale flags are landmines — clean them up.

Expand-contract migration — add new schema, migrate, drop old.
 ▲ Never rename/drop in one step; backward-compatible stages or you break in-flight code.

MESSAGING & QUEUES

At-least / at-most / exactly-once — delivery guarantees.
 ▲ At-least-once is the real default; "exactly-once" is at-least-once + dedup, never magic.

Dead-letter queue (DLQ) — sink for messages that keep failing.
 ▲ Without one, a poison message blocks the queue forever; alert on DLQ depth.

Consumer lag — how far behind consumers are vs producers.
 ▲ The key health signal; rising lag = you're losing — scale consumers or shed.

Ordering & partitions — order is per-partition, not global.
 ▲ Need global order? You serialize through one partition — that's your throughput cap.

Outbox pattern — write event + data in one txn, relay publishes.
 ▲ Solves dual-write, but delivers at-least-once — consumers still need dedup.

DEBUGGING & ANALYTICS

Request tracing / request ID — correlation ID threaded through every hop.
 ▲ Must propagate across async/queue boundaries or the trace breaks where it matters most.

Distributed tracing — spans across services (OpenTelemetry).
 ▲ 100% sampling is expensive; tail-sample to keep the slow/error traces.

Debug / dynamic log levels — raise verbosity without redeploy.
 ▲ Debug logging in hot paths is its own perf hit; sample & scope it.

Requests-as-data — emit each request as an analytics event.
 ▲ High-cardinality fields explode metric cardinality & cost; pick dimensions carefully.

Structured logging — logs as key-value/JSON, not free text.
 ▲ Free-text logs can't be queried at scale; structure them or they're write-only.

Logs vs metrics vs traces — events · aggregates · request paths.
 ▲ Don't reconstruct metrics from logs at query time — pre-aggregate the hot ones.

COUNTING AT SCALE

Approx counting (batched) — buffer increments, flush aggregated.
 ▲ Unflushed counts lost on crash; reads lag the flush interval — fine for "views," not billing.

Real-time counting — atomic increment per event (Redis).
 ▲ A single hot counter row serializes writes; shard the counter, sum on read.

Probabilistic (HLL / CMS) — distinct/frequency in tiny fixed memory.
 ▲ Bounded error by design; never use where exactness is required.

RATE LIMITING

In-memory (per node) — each node enforces its own limit locally.
 ▲ No global view — N nodes = N× the intended limit; only fair if traffic is even.

Centralized (Redis) — shared atomic counter for a true global limit.
 ▲ Adds a network hop & a hotspot dependency; if it's down, do you fail open or closed?

Full-mesh / gossip — nodes share counts to approximate global.
 ▲ Eventually-consistent counts → brief over-admission; cheaper than centralized.

Token bucket — tokens refill at rate R, burst up to B.
 ▲ Tune B carefully — too large defeats the limit, too small rejects legit bursts.

Sliding window — count over a moving interval, not a fixed reset.
 ▲ Fixed windows allow a 2× burst at the boundary; sliding log is precise but memory-heavy.

Cost-based limiting — charge requests by weight, not count.
 ▲ A flat req/s limit is gameable — one expensive call ≠ one cheap one; price by work.

AUTHENTICATION

Token validation (JWT) — verify signature + claims, no DB hit.
 ▲ Stateless = can't easily revoke; use short TTLs + a refresh/denylist.

JWKS — public-key endpoint for verifying JWT signatures.
 ▲ Cache keys, but honor rotation; a stale JWKS cache rejects valid tokens.

OAuth2 + PKCE — delegated auth; PKCE secures public clients.
 ▲ Don't roll your own; use the auth-code+PKCE flow, never implicit.

mTLS — mutual cert auth for service-to-service.
 ▲ Cert rotation/expiry is the operational trap — automate it or you get outages.

AuthN vs AuthZ — who you are vs what you may do (RBAC/ABAC/ReBAC).
 ▲ Don't conflate them; a valid token still needs a permission check on every action.

Refresh-token rotation — issue a new refresh token on each use.
 ▲ Detects token theft (reuse of an old one); without it a leaked refresh token lives forever.

COSTING & BILLING

No free lunch — every quality (a nine, low latency) has a cost.
 ▲ Each extra nine ≈ 10× spend; match reliability to what the business will pay for.

Cost per customer at scale — unit economics of serving one user.
 ▲ Watch the heavy tail — a few power users can dominate cost; meter & cap them.

Pay-per-request pricing — bill per call when cost is predictable.
 ▲ Breaks if request cost varies wildly; weight by actual work or you lose money.

Pay-per-resource pricing — bill by compute/storage/egress consumed.
 ▲ Egress is the silent budget-killer; attribute & alert on it (FinOps).

Compute isolation — separate the execution environment per tenant (VM, dedicated node, namespace+cgroups).

⚠ Gov/HIPAA workloads often can't share a kernel/host; shared containers may fail audit — use VMs or dedicated nodes.

Storage / DB isolation — silo data per tenant: row (shared), schema, or database-per-tenant.

⚠ Shared-row needs flawless tenant-ID filtering — one missing WHERE clause = a cross-tenant leak & breach.

Network isolation — per-tenant VPC/subnet, security groups, no shared egress.

⚠ "Soft" isolation (just app-layer) rarely satisfies regulators; they want network-level proof.

Noisy neighbor — one tenant's load degrades others on shared infra.

⚠ The hidden cost of shared isolation; needs quotas + per-tenant rate limits to contain.

Data residency — keep a tenant's data in a specific region/jurisdiction.

⚠ Backups, logs & caches also count — residency leaks through the side doors you forget.

Single-tenant vs pooled — dedicated stack vs shared with logical separation.

⚠ Single-tenant is the safe audit answer but multiplies cost & ops; tier it by customer.

GDPR — EU data-protection law: consent, access, and the right to be forgotten.

⚠ Applies to any service touching EU residents' data, wherever you're hosted; fines scale with revenue.

On-demand data deletion — purge a user's data across all stores on request (GDPR/CCPA "right to erasure").

⚠ Hard with backups, caches, logs, analytics & replicas — design deletion paths up front, don't bolt on.

HIPAA — US health-data rules: protect PHI, audit access, sign BAAs with vendors.

⚠ Encryption + access logs aren't optional; every sub-processor needs a BAA or you're non-compliant.

PCI-DSS — payment-card security standard for handling cardholder data.

⚠ Tokenize/outsource card data to shrink scope — storing PANs yourself makes the whole system in-scope.

SOC 2 — audited report on security/availability controls (trust for B2B).

⚠ It's about evidence: if it isn't logged & documented, the auditor treats it as not done.

Data classification — tag data by sensitivity (public → PII → PHI/PCI).

⚠ You can't protect what you haven't classified; controls & retention follow the tag.

Audit trail — immutable, tamper-evident log of who accessed/changed what.

⚠ Required by most regimes; if it's mutable or gappy, it won't hold up in an audit.

SRE — Site Reliability Engineering; run ops with software & SLOs, not toil.
 ▲ If your "SRE" team just does manual ops, you've renamed sysadmins — automate or it doesn't scale.
Toil — manual, repetitive, automatable work with no lasting value.
 ▲ Toil grows linearly with scale; cap it (e.g. <50% of time) or the team drowns.
On-call / runbook — rotation that responds; runbook = step-by-step fix guide.
 ▲ Alerts without runbooks just wake people up; every page should be actionable & documented.
Incident severity (SEV) — SEV1 (critical) → SEV3+; sets response & comms.
 ▲ Under-classifying delays the right response; over-classifying causes alert fatigue.
MTTD / MTTR / MTBF — mean time to detect / recover / between failures.
 ▲ MTTR usually beats MTBF to improve — you can't prevent all failures, so recover fast.
Blameless postmortem — write-up of cause & fixes, no finger-pointing.
 ▲ Blame hides the truth; people stop reporting. Fix systems & processes, not people.
Alert fatigue — too many low-value alerts dull response.
 ▲ Noisy alerts get ignored — then the real one is missed; alert on symptoms, not every cause.
Change freeze — halt deploys during peak/risk windows.
 ▲ Freezes batch up risk into one big post-freeze release; prefer small continuous over big-bang.
Capacity / DR drill — rehearse failover & load before you need it.
 ▲ An untested backup/failover is a hope, not a plan — drill it or it fails when it counts.
Golden signals — latency, traffic, errors, saturation: what to watch.
 ▲ Dashboards full of vanity metrics hide these four; alert on the signals, not CPU graphs.

CapEx vs OpEx — upfront capital (own hardware) vs ongoing spend (cloud).
 ▲ Cloud shifts CapEx → OpEx; great for flexibility, but unmanaged OpEx creeps without a ceiling.
TCO — total cost of ownership: build + run + people + opportunity.
 ▲ "Free"/cheap options often have high TCO via ops burden; count the whole lifecycle.
Unit economics — cost & revenue per unit (user, request, GB).
 ▲ If cost-per-unit doesn't drop with scale, growth makes losses bigger, not smaller.
Gross margin — revenue minus cost of serving it (infra, support).
 ▲ Cloud/egress/LLM spend eats margin silently; track margin per product line, not just topline.
COGS — cost of goods sold; for SaaS = the cost to run the service.
 ▲ Misclassifying R&D as COGS (or vice-versa) distorts margin & valuation.
Burn rate / runway — monthly net spend / months of cash left.
 ▲ A big cloud commit can shorten runway fast; model spend against runway before signing.
Reserved / spot / committed-use — discounts for upfront or interruptible capacity.
 ▲ Over-commit and you pay for idle; spot can vanish mid-job — only for fault-tolerant work.
Chargeback / showback — bill (or just show) teams their cloud spend.
 ▲ Untagged resources can't be attributed; without tagging discipline, FinOps is blind.
ROI / payback period — return vs cost; time to recoup an investment.
 ▲ Migrations/rewrites often have long payback; sponsors lose patience — phase the value.
Build vs buy — in-house vs vendor/SaaS for a capability.
 ▲ Build only your differentiator; buy the rest, or you spend headcount reinventing commodities.